FADENA
FACULTAD DE DEFENSA NACIONAL

Lenguajes de Programación

Clase 9: POO Avanzada

Docentes:
- Ing. Sebastián Balestrini
- Ing. Gustavo Distéfano

Objetivos previos

- Comprender en qué se diferencia la POO del paradigma estructurado.
- Conocer los conceptos básicos de clase, objeto, atributos y métodos.
- Identificar la relación entre modelo del mundo real y objetos en programación.
- Comprender conceptos básicos del modelado UML
- Introducción a los lenguajes de programación C++ y Java
- Comprender los principios de encapsulamiento, herencia y polimorfismo.
- Analizar cómo estos principios ayudan a organizar y reutilizar código.
- Aprender a representar relaciones entre clases mediante un diagrama UML básico.
- Ver ejemplos básicos de estos conceptos en C++ y Java.

Objetivos clase 9

- Aplicar los conceptos fundamentales de la programación orientada a objetos en el modelado de un sistema real simplificado.
- Diseñar un modelo de clases que incluya relaciones de asociación, composición y herencia entre objetos.
- Identificar e implementar encapsulamiento a través del uso adecuado de modificadores de visibilidad (public, private, protected).
- Representar clases, atributos, métodos y relaciones mediante un diagrama UML claro y completo.
- Comprender cómo se instancian objetos y se relacionan entre sí en un sistema.
- Desarrollar la capacidad de traducir un requerimiento funcional en estructuras de clases y métodos.
- Reconocer cómo la POO permite escalar y mantener sistemas mediante la reutilización de código y la organización modular.

Herramientas

- Lucidchart: https://www.lucidchart.com
- Online GDB: https://www.onlinegdb.com/

RESUMEN

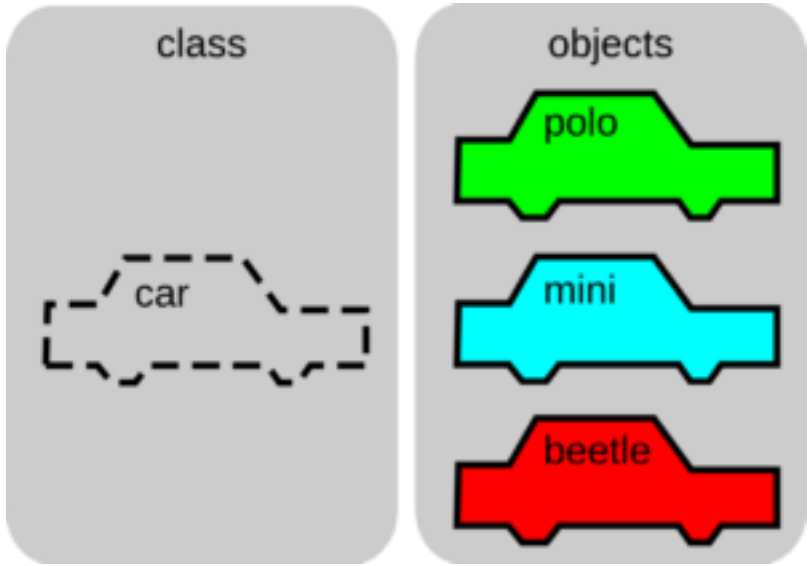
Programación Orientada a Objetos (POO)

El paradigma orientado a objetos es útil cuando el sistema se modela de forma casi análoga a la realidad, porque así se simplifica el diseño de alto nivel. Esta analogía permite que los programadores tengan más claro cuál es el papel de cada porción del programa y de los datos, lo que facilita la creación y el mantenimiento del sistema. Además, se promueve la reutilización, pues las similitudes entre objetos se programan sólo una vez en forma abstracta y el programador concentra su esfuerzo en las diferencias concretas.

Clases e Instancias

Una **clase** representa todas las propiedades (atributos) y comportamientos(métodos) de un objeto.

Para referirnos al objeto de una clase usamos el termino **instancia**.



Encapsulamiento

- Oculta atributos y expone solo métodos de acceso
- Mejora la organización y seguridad del código

Herencia

- Una clase hija reutiliza atributos y métodos de una clase padre
- Evita duplicar código y permite extender funcionalidades

Polimorfismo

- Un mismo método se comporta diferente según el objeto

UML

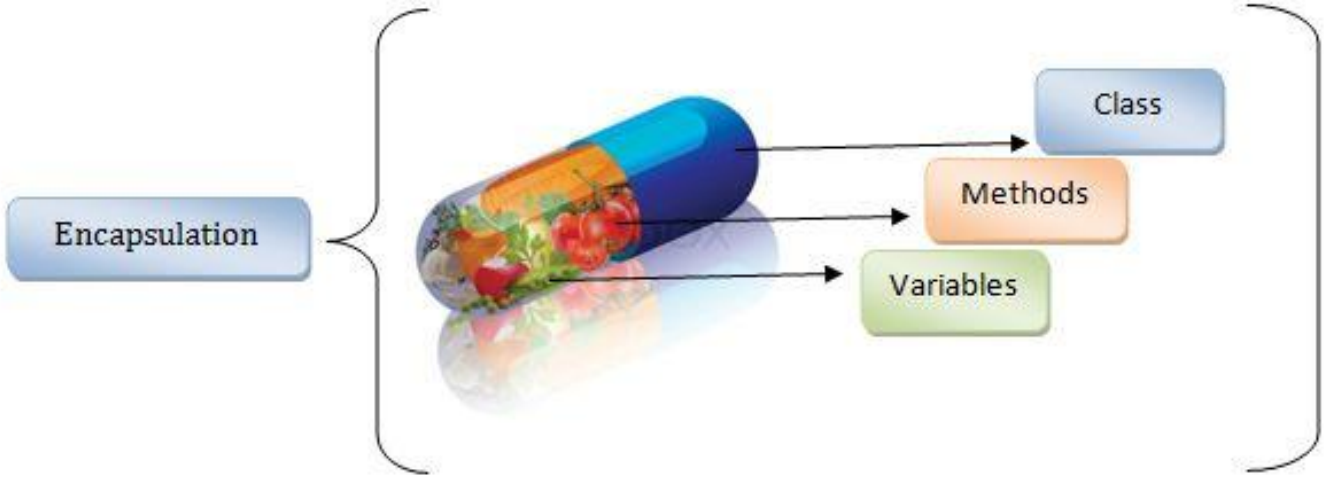
- Visualiza clases, atributos, métodos y relaciones
- Flecha con triángulo vacío = herencia

Principios de la Programación Orientada a Objetos

“La POO organiza mejor el código mediante principios clave que facilitan reutilización, mantenimiento y legibilidad.”

Encapsulamiento

- Ocultar los detalles internos de una clase y exponer solo lo necesario.
- Protege los atributos usando modificadores de acceso.
- Permite mantener control sobre los datos.

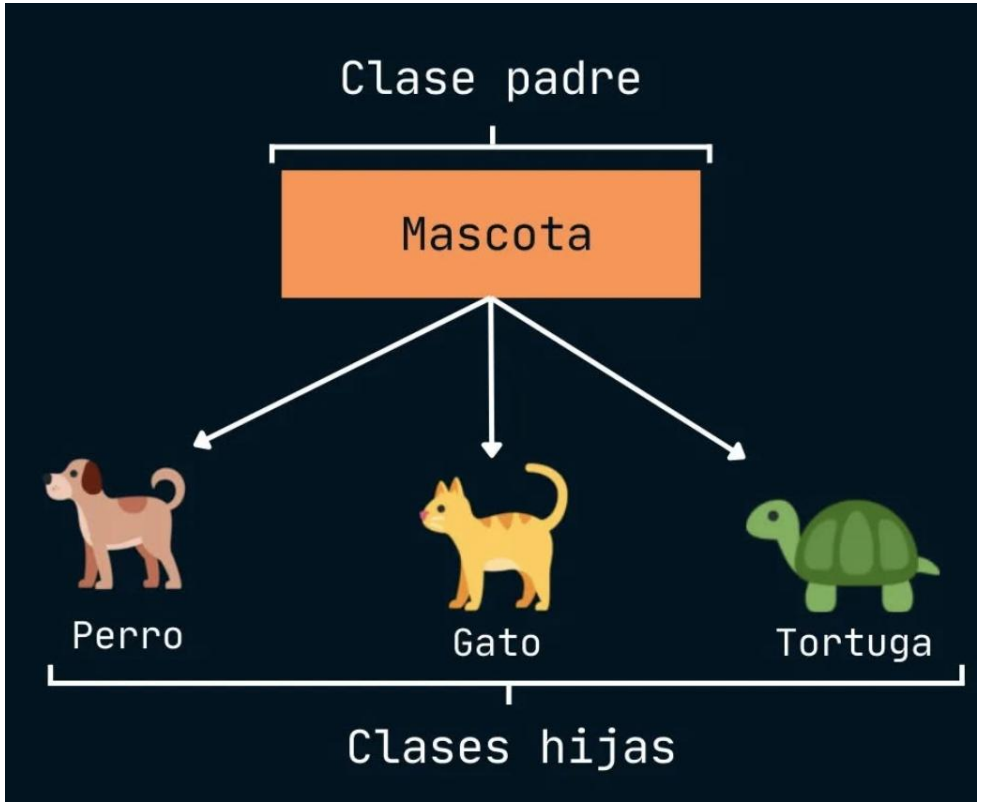


Principios de la Programación Orientada a Objetos

Herencia

Al implementar una clase y para reutilizar el código una clase puede extender de otra, heredando el comportamiento de la clase extendida. La relación de herencia entre las clases es una relación de «es-un».

- Una clase hija hereda atributos y métodos de una clase padre.
- Permite reutilizar código y extender comportamientos.

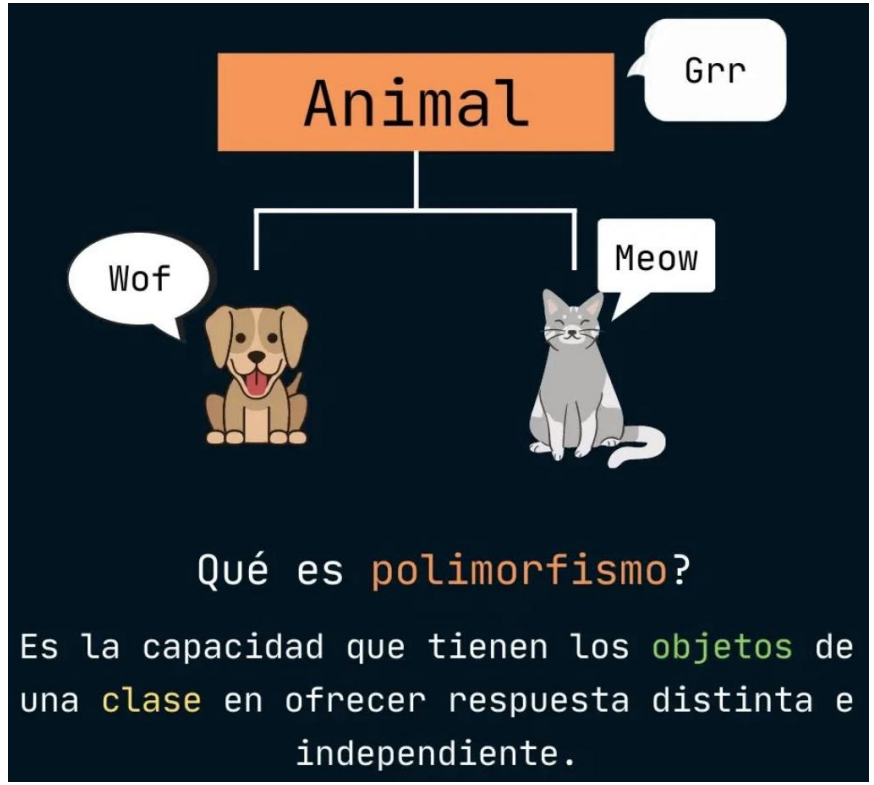


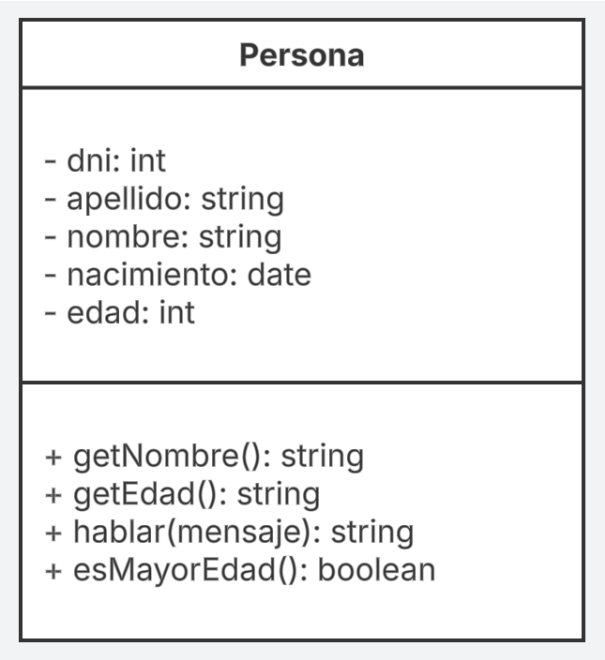
FADENA
FACULTAD DE DEFENSA NACIONAL

Principios de la Programación Orientada a Objetos

Polimorfismo

- Un mismo método se comporta diferente según el objeto que lo use.
- Permite sobrescribir métodos en clases hijas.





- **- private**: Solo accesible dentro de la misma clase. Ideal para proteger datos sensibles.
- **# protected**: Accesible desde la clase y sus subclases. Útil en herencia controlada.
- **+ public**: Accesible desde cualquier clase. Usado para métodos públicos y atributos visibles.

Modificador	**Acceso desde la misma clase**	**Subclases**	**Otras clases**
- private	✓ Sí	✗ No	✗ No
#protected	✓ Sí	✓ Sí	✗ No
+public	✓ Sí	✓ Sí	✓ Sí

Ejercicio

Objetivo de la actividad

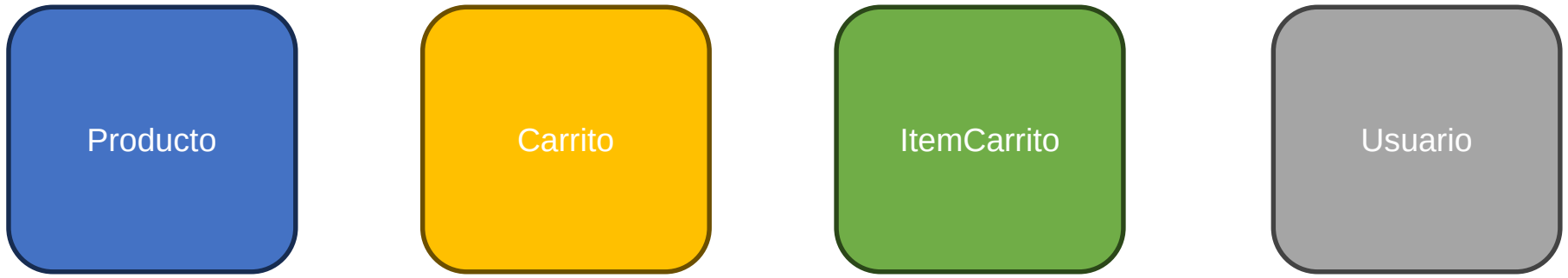
Modelar un sistema simplificado de carrito de compras utilizando programación orientada a objetos, con foco en encapsulamiento, relaciones entre clases y lógica de negocio.

Funcionalidades del sistema

El sistema debe permitir:

- Listar productos disponibles (con nombre, precio, stock).
- Agregar productos al carrito, indicando cantidad.
- Ver el carrito con los productos seleccionados y sus cantidades.
- Incrementar o decrementar la cantidad de un producto en el carrito.
- Eliminar un producto del carrito.
- Finalizar la compra, mostrando el total a pagar y vaciando el carrito.

¿Clases del sistema?



Producto

- id: int
- nombre: String
- precio: float
- stock: int

ItemCarrito

- producto: Producto
- cantidad: int
- subtotal(): float → devuelve cantidad * producto.precio

Carrito

- items: List<ItemCarrito>
- agregarProducto(producto, cantidad)
- eliminarProducto(producto)
- modificarCantidad(producto, nuevaCantidad)
- mostrarCarrito()
- total(): float
- vaciarCarrito()

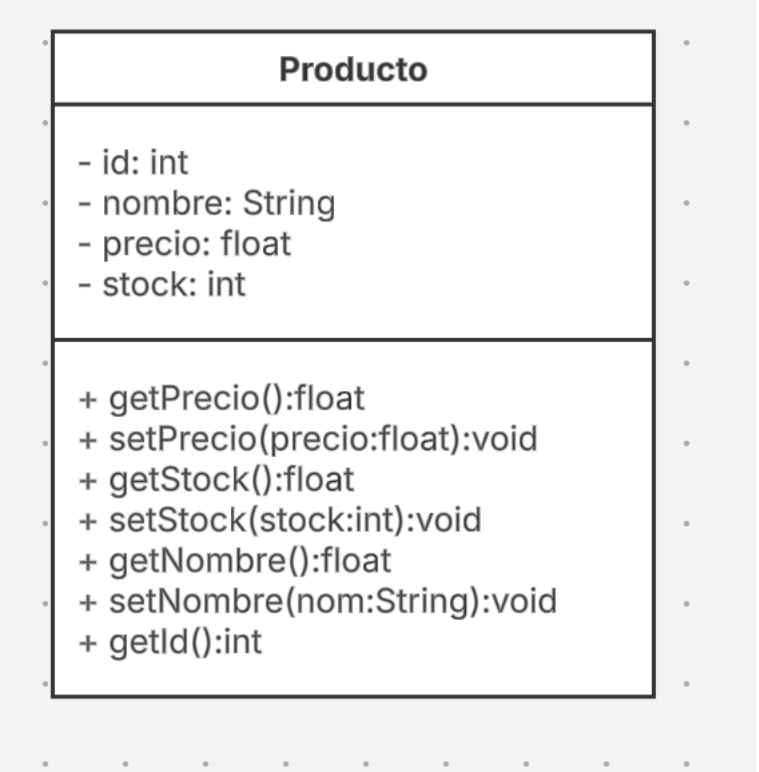
Usuario

- nombre: String
- carrito: Carrito

¿Modelamos?

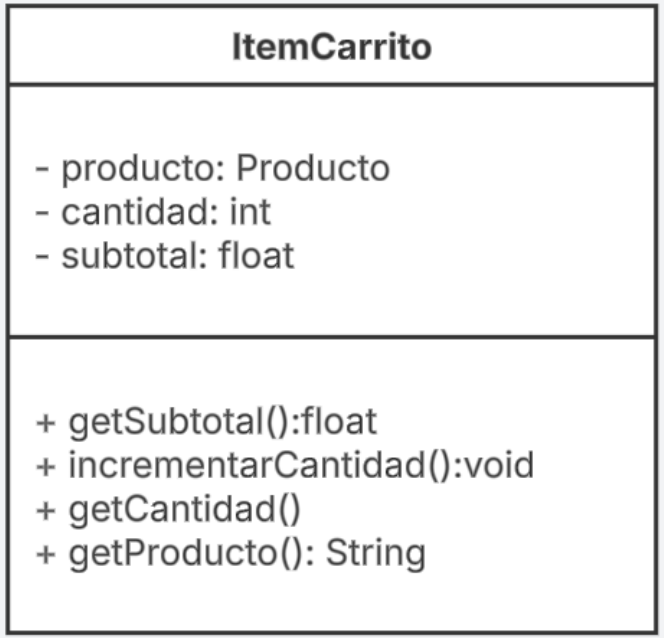
Producto

- id: int
- nombre: String
- precio: float
- stock: int



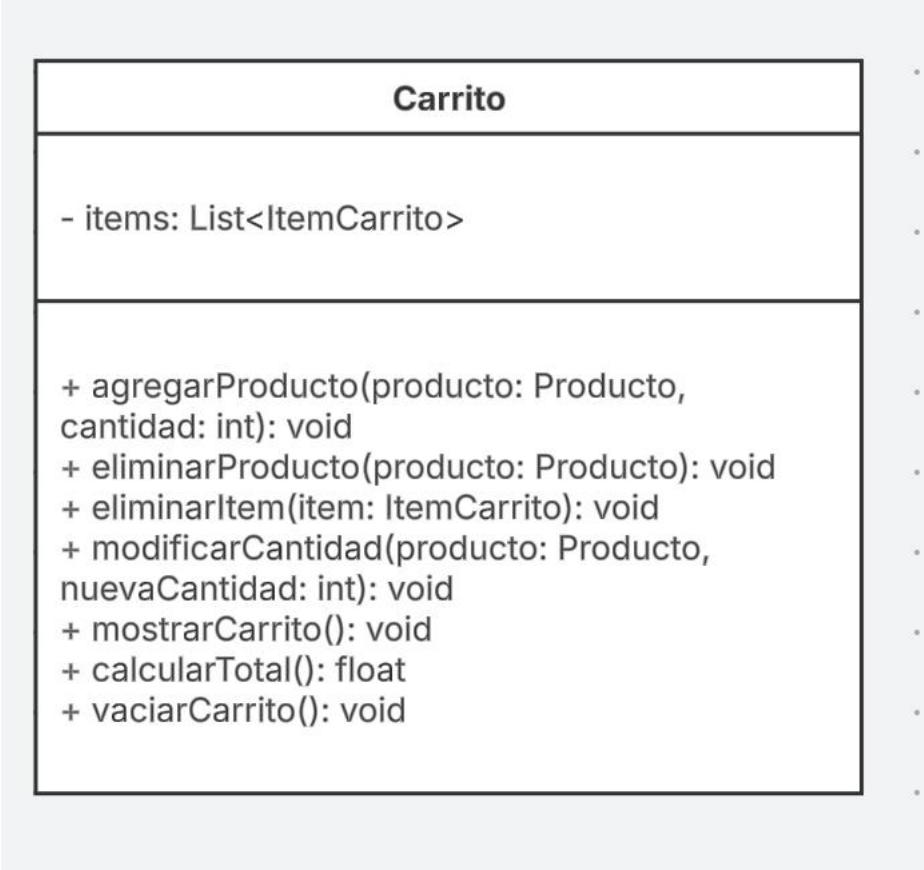
ItemCarrito

- producto: Producto
- cantidad: int
- subtotal(): float → devuelve cantidad * producto.precio



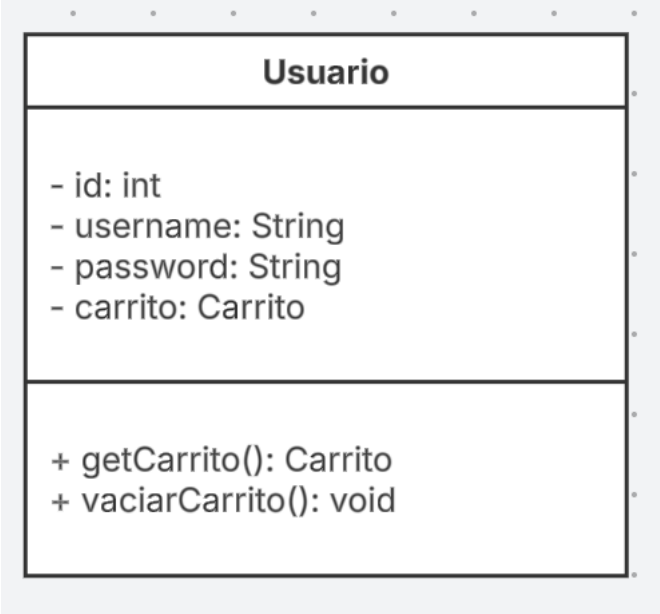
Carrito

- items: List<ItemCarrito>
- agregarProducto(producto, cantidad)
- eliminarProducto(producto)
- modificarCantidad(producto, nuevaCantidad)
- mostrarCarrito()
- total(): float
- vaciarCarrito()



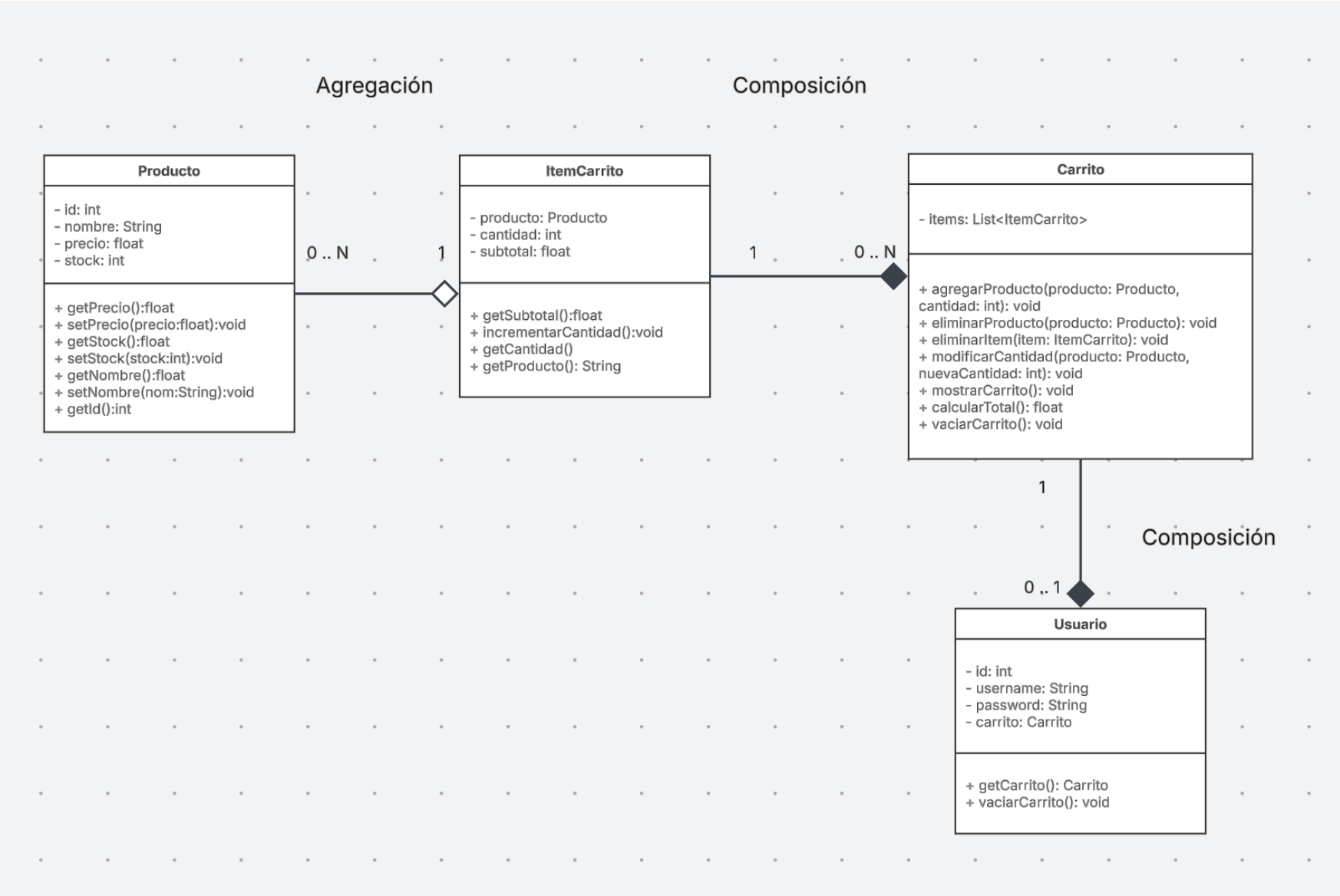
Usuario

- nombre: String
- carrito: Carrito



Relaciones

Relaciones


Agregación
Composición
Producto
- id: int
- nombre: String
- precio: float
- stock: int
+ getPrecio():float
+ setPrecio(precio:float):void
+ getStock():float
+ setStock(stock:int):void
+ getNombre():float
+ setNombre(nom:String):void
+ getId():int
0 .. N
1
ItemCarrito
- producto: Producto
- cantidad: int
- subtotal: float
+ getSubtotal():float
+ incrementarCantidad():void
+ getCantidad()
+ getProducto(): String
1
0 .. N
Carrito
- items: List<ItemCarrito>
+ agregarProducto(producto: Producto, cantidad: int): void
+ eliminarProducto(producto: Producto): void
+ eliminarItem(item: ItemCarrito): void
+ modificarCantidad(producto: Producto, nuevaCantidad: int): void
+ mostrarCarrito(): void
+ calcularTotal(): float
+ vaciarCarrito(): void
1
Composición
0 .. 1
Usuario
- id: int
- username: String
- password: String
- carrito: Carrito
+ getCarrito(): Carrito
+ vaciarCarrito(): void

¿ Dudas ?

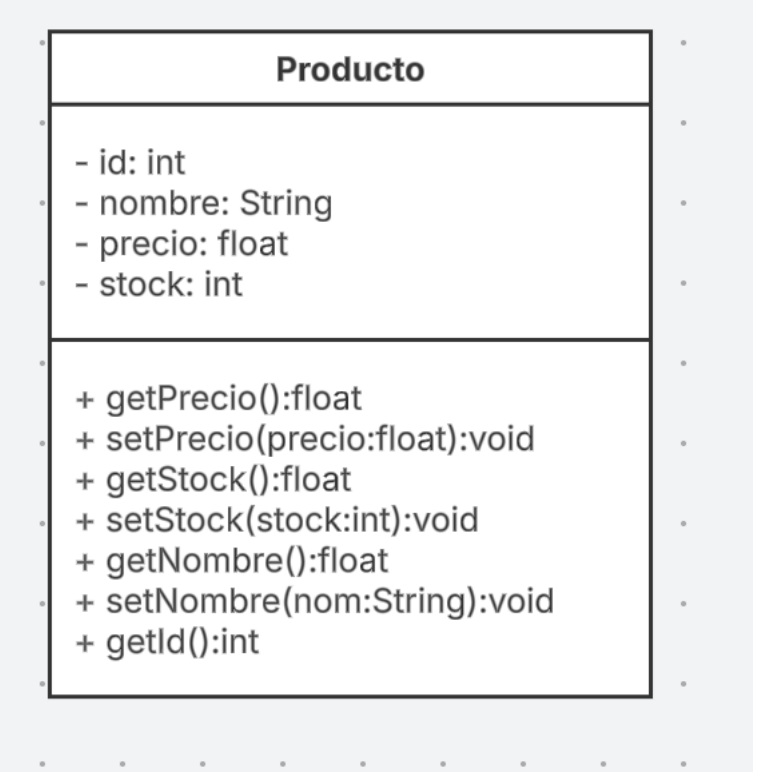
```java
class Producto {
    private int id;
    private String nombre;
    private float precio;
    private int stock;

    public Producto(int id, String nombre, float precio, int stock) {
        this.id = id;
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
    }

    public int getId() { return id; }
    public String getNombre() { return nombre; }
    public float getPrecio() { return precio; }
    public int getStock() { return stock; }

    public void setPrecio(float precio) { this.precio = precio; }
    public void setStock(int stock) { this.stock = stock; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public void mostrarProducto() {
        System.out.println("Producto: " + nombre + ", Precio: $" + precio + ", Stock: " + stock);
    }
}
```



```java
class ItemCarrito {
    private Producto producto;
    private int cantidad;
    private float subtotal;

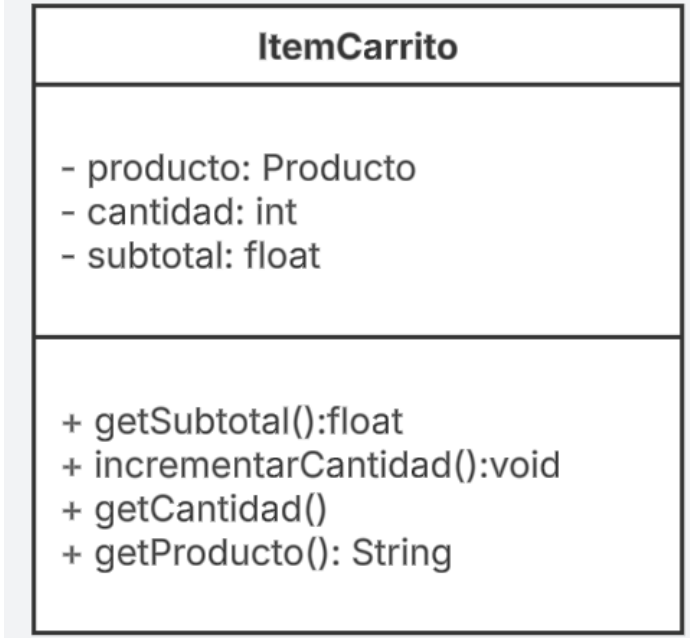
    public ItemCarrito(Producto producto, int cantidad) {
        this.producto = producto;
        this.cantidad = cantidad;
        this.subtotal = calcularSubtotal();
    }

    private float calcularSubtotal() {
        return producto.getPrecio() * cantidad;
    }

    public float getSubtotal() { return subtotal; }
    public void incrementarCantidad() {
        cantidad++;
        subtotal = calcularSubtotal();
    }

    public int getCantidad() { return cantidad; }
    public String getProducto() { return producto.getNombre(); }

    public void mostrarItem() {
        System.out.println("- " + producto.getNombre() + " x" + cantidad + " → $" + subtotal);
    }
}
```



```java
class Carrito {
    private ArrayList<ItemCarrito> items;

    public Carrito() {
        items = new ArrayList<>();
    }

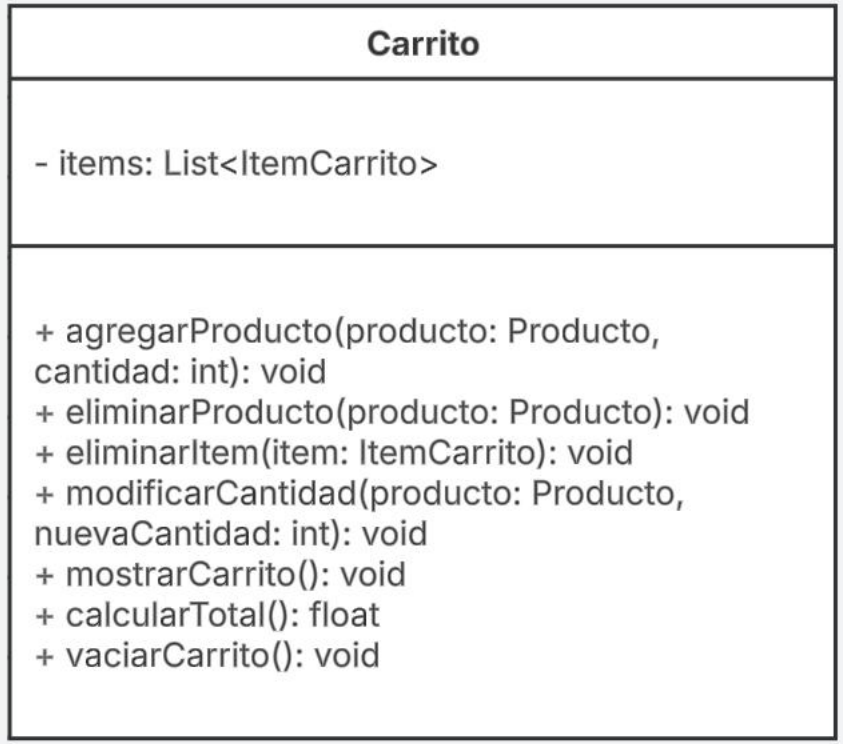
    public void agregarProducto(Producto producto, int cantidad) {
        for (ItemCarrito item : items) {
            if (item.getProducto().equals(producto.getNombre())) {
                // Si ya está, incrementa cantidad
                item.incrementarCantidad();
                return;
            }
        }
        items.add(new ItemCarrito(producto, cantidad));
    }

    public void eliminarProducto(Producto producto) {
        items.removeIf(item -> item.getProducto().equals(producto.getNombre()));
    }

    public void vaciarCarrito() {
        items.clear();
    }

    public float total() {
        float total = 0;
        for (ItemCarrito item : items) {
            total += item.getSubtotal();
        }
        return total;
    }

    public void mostrarCarrito() {
        System.out.println("🛒 Carrito de compras:");
```



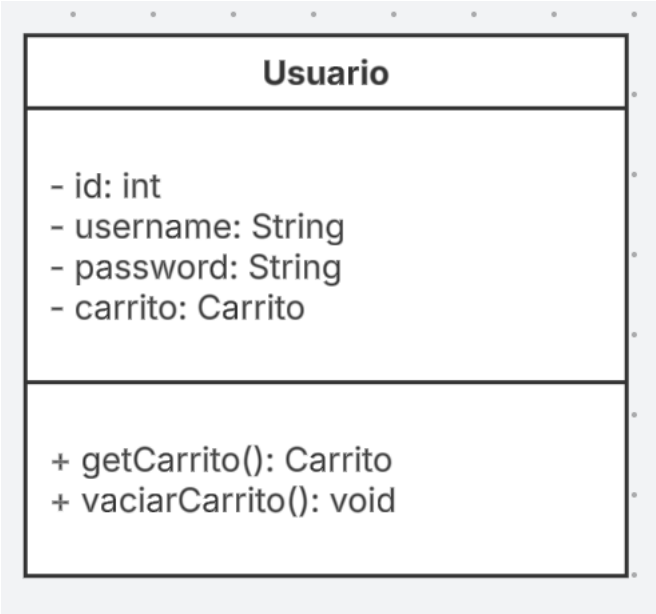
```java
class Usuario {
    private String nombre;
    private String email;
    private Carrito carrito;

    public Usuario(String nombre, String email) {
        this.nombre = nombre;
        this.email = email;
        this.carrito = null; // No tiene carrito al inicio
    }

    public void iniciarCarrito() {
        if (carrito == null) {
            carrito = new Carrito();
            System.out.println("🛒 Carrito iniciado para " + nombre);
        } else {
            System.out.println("Ya tienes un carrito activo.");
        }
    }

    public void agregarAlCarrito(Producto producto, int cantidad) {
        if (carrito == null) {
            System.out.println("Debes iniciar tu carrito primero.");
        } else {
            carrito.agregarProducto(producto, cantidad);
        }
    }

    public void verCarrito() {
        if (carrito != null) {
            carrito.mostrarCarrito();
        } else {
            System.out.println("No tienes un carrito activo.");
        }
    }
```



```java
public class Main
{
    public static void main(String[] args) {
        Producto prod1 = new Producto(1, "Mouse", 45000.99f, 10);
        Producto prod2 = new Producto(2, "Teclado", 75000.70f, 5);
        Producto prod3 = new Producto(1, "Memoria USB", 120000.00f, 5);

        prod1.mostrarProducto();
        prod2.mostrarProducto();
        prod3.mostrarProducto();
        // Cambiamos el stock del producto 1
        prod1.setStock(8);
        System.out.println("Stock actualizado de " + prod1.getNombre() + ": " + prod1.getStock());

        Usuario usuario = new Usuario("Ana", "ana@example.com");
        usuario.verCarrito(); // No tiene aún
        usuario.iniciarCarrito();
        usuario.agregarAlCarrito(prod1, 1);
        usuario.agregarAlCarrito(prod2, 2);
        usuario.verCarrito();
        usuario.agregarAlCarrito(prod1, 3);
        usuario.verCarrito();
        usuario.finalizarCompra();

    }
}
```

```
Producto: Mouse, Precio: $45000.99, Stock: 10
Producto: Teclado, Precio: $75000.7, Stock: 5
Producto: Memoria USB, Precio: $120000.0, Stock: 5
Stock actualizado de Mouse: 8
No tienes un carrito activo.
🛒Carrito iniciado para Ana
🛒Carrito de compras:
- Mouse x1 -> $45000.99
- Teclado x2 -> $150001.4
Total: $195002.39
🛒Carrito de compras:
- Mouse x2 -> $90001.98
- Teclado x2 -> $150001.4
Total: $240003.38
Compra finalizada. Total a pagar: $240003.38


...Program finished with exit code 0
Press ENTER to exit console.
```

FADENA
FACULTAD DE DEFENSA NACIONAL

Lenguajes de Programación

Clase 9: POO Avanzada

Docentes:
- Ing. Sebastián Balestrini
- Ing. Gustavo Distéfano